TECHNISCHE UNIVERSITÄT
CHEMNITZ

Professur Betriebssysteme

Algorithmen und Programmierung, WS 2025/2026

# Belegaufgabe #6: **CPU**

Deadline: **19.01.2026, 23.59 Uhr**

Erreichbare Punkte: **10**

## 1. Motivation

Ziel dieser Aufgabe ist…

- …das Verstehen und Umsetzen einer nichttrivialen textuellen Spezifikation;
- …der Umgang mit Bitoperatoren und `struct`s;
- …der Umgang mit Konzepten wie Komplement oder Endianess;
- …der korrekte Umgang mit Speicher und Arrays.

## 2. Harward-Simulator

Die Harvard-Architektur[1] zeichnet sich dadurch aus, dass Befehls- und Datenspeicher weitgehend getrennt sind. In dieser Aufgabe geht es um einen fiktiven (stark ereinfachten) 4-Bit-Rechner mit einem Adressraum (sowohl der des Programm- als auch der des Datenspeichers) von jeweils 8 Bit. Sie sollen dafür einen Simulator schreiben, der Änderungen im Speicher zurückgibt.

### Architektur

Der Rechner hat drei Register, siehe Abbildung 1:

- **Befehlszeiger PC**, 8 Bit, soll stets die Adresse des nächsten zu interpretierenden Befehls beinhalten (wenn keine Sprünge vorkommen)[2]
- **Kombiregister**, 8 Bit, besteht aus den Flags **Z** (Zero, für Null) und **C** (Carry, für Übertrag) in den Bits 7 und 6, sowie aus dem 4-Bit-Akkumulator (**A**) in den Bits 0 bis 3. Der Akkumulator sammelt die Ergebnisse einer Befehlsabarbeitung, kann aber auch selbst Operand sein. Die Bits 4 und 5 werden nicht genutzt und dürfen nicht verändert werden.

[1] `https://de.wikipedia.org/wiki/Harvard-Architektur`

[2] also des Befehls, der dem aktuellen folgt, aufsteigend

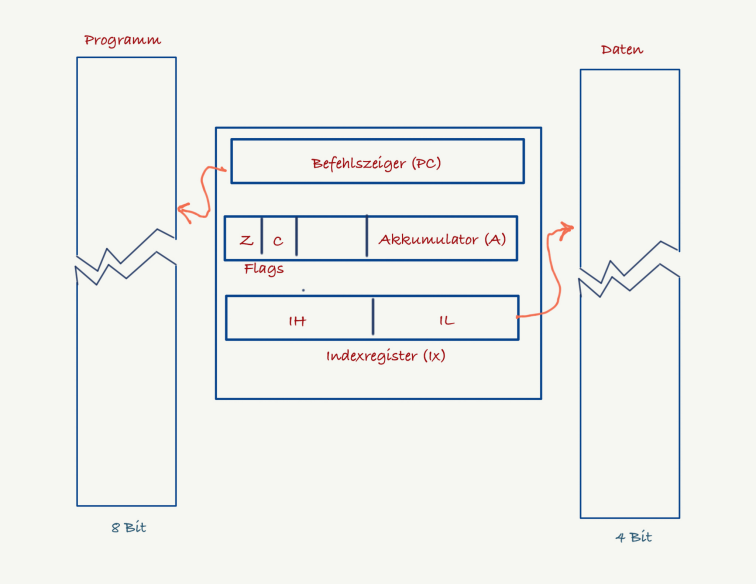


Abbildung 1: 4-Bit „Rechner“

- **Indexregister IX**, 8 Bit, dient vor allem zur indirekten Adressierung. Die jeweils höhere (**IH**) und niedrigere Hälfte (**IL**) können jedoch als einzelne 4-Bit-Register angesprochen werden.

### Befehlsabarbeitung

Der Rechner führt den Standardzyklus der Befehlsabarbeitung aus:

1. Er liest einen Befehl aus dem Programmspeicher, dessen Adresse durch den Befehlszeiger PC gegeben ist.
2. Er erhöht den Befehlszeiger um 1, sodass er auf den nächsten Befehl zeigt.
3. Anschließend führt er den gelesenen Befehl aus und ändert dabei ggf. Register, Flags oder Speicher.
4. Gehe zu Schritt 1.

Ein Befehl des Rechners besteht aus jeweils einem 8-Bit-Datenwort. Damit man dessen Sinn leichter erfassen kann, werden Befehlsmuster auch mit Kürzeln (sogenannten Mnemonics) dargestellt, denen ggf. ein oder zwei Argumente folgen. Bei zwei Argumenten (das kommt nur bei Transfer vor) steht das erste immer für das Ziel der Operation, das zweite für die Quelle.

In Tabelle 1 ist der Befehlssatz mit allen Befehlen, ihren Bitmustern und einer Beschreibung ihres Verhaltens aufgelistet. Das Bitmuster eines Befehls besteht dabei aus mehreren Teilen. Diese dienen zur Identifikation sowie zur Beschreibung der Parameter des Befehls. Die Spalte „Flags“ gibt durch ein × an, ob durch die Befehlsausführung die

jeweiligen Flags geändert werden können. Ist das der Fall, so wird das Zero-Flag gesetzt[3] wenn das Ergebnis 0 entspricht. Ansonsten wird es zurückgesetzt. Das Carry-Flag wird bei Additionsoperationen auf 1 gesetzt, wenn das Ergebnis „überläuft“, also ein füntes Bit zur Ergebnisdarstellung benötigt werden würde. Bei Additionsoperationen bei denen dies nicht der Fall ist, wird das Flag auf 0 gesetzt.

Die Tabellen 2 und 3 geben an, wie die Argumente bzw. die Bedingungen für die bedingten Sprünge kodiert sind.

Tabelle 1: Befehlssatz des Rechners

| **Mnemonic/ Bedeutung** | **Bitmuster** | Flags Z | Flags C | **Wirkung** |
|---|---|---|---|---|
| `ldi` *adr*, *val*<br>*load immediate* | 7 6 5 4 3 … 0<br>0 0 \| *adr* \| *val* | | | Lädt den im Befehl enthaltenen 4-Bit-Wert *val* an die durch *adr* angegebene Stelle. |
| `mv` $adr_1$, $adr_2$<br>*move* | 7 … 4 3 2 1 0<br>0 1 0 0 \| $adr_1$ \| $adr_2$ | | | Kopiert den Wert in $adr_2$ nach $adr_1$. |
| `jr` *dist*<br>*jump relative* | 7 … 4 3 … 0<br>0 1 0 1 \| *dist* | | | Erhöht den Wert von PC um den Wert *dist*. Dabei ist *dist* im Zweierkomplement. |
| `halt`<br>*halt execution* | 7 … 0<br>0 1 1 1 1 1 1 1 | | | Hält die Ausführung des Rechners an. |
| `b` *cc*, *dist*<br>*branch when condition* | 7 6 5 4 3 … 0<br>1 1 \| *cc* \| *dist* | | | Erhöht bei erfüllter Bedingung (siehe Tabelle 3) den Wert von PC um den Wert *dist*. Dabei ist *dist* im Zweierkomplement. |
| `add` *adr*<br>*add* | 7 … 2 1 0<br>1 0 1 0 0 0 \| *adr* | × | × | Addiert die Werte im Akkumulator und in *adr* und speichert das Ergebnis im Akkumulator. |
| `adc` *adr*<br>*add with carry* | 7 … 2 1 0<br>1 0 1 0 0 1 \| *adr* | × | × | Addiert die Werte im Akkumulator, in *adr* sowie das Übertragsflag (C) und speichert das Ergebnis im Akkumulator. |
| `and` *adr*<br>*and* | 7 … 2 1 0<br>1 0 1 0 1 0 \| *adr* | × | | Verküpft den Wert im Akkumulator und den Wert aus *adr* bitweise mit UND und speichert das Ergebnis in Akkumulator. |

[3] erhält den Wert eins

Tabelle 1: Befehlssatz des Rechners (Fortsetzung)

| **Mnemonic/ Bedeutung** | **Bitmuster** | Flags Z | Flags C | **Wirkung** |
|---|---|---|---|---|
| `or` *adr*<br>*or* | 7 … 2 1 0<br>1 0 1 0 1 1 \| *adr* | × | | Verküpft den Wert im Akkumulator und den Wert aus *adr* bitweise mit ODER und speichert das Ergebnis in Akkumulator. |
| `xor` *adr*<br>*exclusive or* | 7 … 2 1 0<br>1 0 1 1 0 0 \| *adr* | × | | Verküpft den Wert im Akkumulator und den Wert aus *adr* bitweise mit Exklusiv-ODER und speichert das Ergebnis in Akkumulator. |
| `neg` *adr*<br>*negation* | 7 … 2 1 0<br>1 0 1 1 0 1 \| *adr* | × | | Bildet vom Wert in *adr* das Einerkomplement und speichert das Ergebnis in Akkumulator. |
| `cpl` *adr*<br>*complement* | 7 … 2 1 0<br>1 0 1 1 1 0 \| *adr* | × | | Bildet vom Wert in *adr* das Zweierkomplement und speichert das Ergebnis im Akkumulator. |

Tabelle 2: Kodierung der Adressen der Argumente (*adr*) in den Befehlen. Es wird der Inhalt des entsprechenden Registers oder der Speicherstelle als Adresse für den Befehl genutzt.

| **Bits** | **Kürzel** | **adressierter Wert** |
|---|---|---|
| `0 0` | `a` | Inhalt des Akkumulators (registerdirekte Adressierung) |
| `1 0` | `ih` | Inhalt des Registers `IH` (registerdirekte Adressierung) |
| `0 1` | `il` | Inhalt des Registers `IL` (registerdirekte Adressierung) |
| `1 1` | `(ix)` | Inhalt der Speicherstelle, deren Adresse in `IX` steht (indirekte Adressierung) |

Tabelle 3: Kodierung der Bedingungen *cc* in bedingten Sprüngen `b` *cc*, *dist*.

| **Bits** | **Kürzel** | **Bedingung** |
|---|---|---|
| `0 0` | `nz` | Das Zero-Flag ist nicht gesetzt ($Z = 0$) |
| `0 1` | `z` | Das Zero-Flag ist gesetzt ($Z = 1$) |
| `1 0` | `nc` | Das Übertragsflag ist nicht gesetzt ($C = 0$) |
| `1 1` | `c` | Das Übertragsflag ist gesetzt ($C = 1$) |

## 3. Aufgabe

Ihre Aufgabe ist es, einen Simulator für die beschriebene Architektur in einer Datei `cpu.rs` zu implementieren. Sie bekommen von uns eine Datei `cpu.rs` mit den entsprechenden Definitionen und ihrer Bedeutung bereitgestellt unter

`https://osg.informatik.tu-chemnitz.de/lehre/aup/belege?file=cpu.rs`

Gegeben sei das Struct `Computer`, welches die beschriebene Architektur darstellt. Die `struct`-Felder von `Computer` haben die folgende Bedeutung:

- `registers`: Datenstruktur, die die Register des Rechners beschreibt.
- `data`: Ein `u8`-Array der Größe 128, das den Datenspeicher darstellt. Beachten Sie, dass jedes Element des Arrays für **zwei** Speicherstellen steht, da ein Speicherwort ja nur vier Bits groß ist. Dabei stehen die oberen (höherwertigen) vier Bits für die größere Adresse.[4]
- `prog`: Ein `u8`-Array der Größe 256, das den Befehlsspeicher darstellt.

Zustätzlich ist der folgende Trait `Cpu` definiert. Er soll für das Struct `Computer` implementiert werden.

```
trait Cpu {
    fn sim_cpu(&mut self) -> Vec<(usize, u8)>;
}
```

Die Methode `simcpu()` soll dabei solange Befehle ausführen, bis entweder

- der Befehl `halt` ausgeführt wird *oder*
- ein ungültiger Befehlscode gefunden wird *oder*
- einhundert (100) Befehle ausgeführt wurden.

Nach der Ausführung von `simcpu()` sollen `registers` und `data` der Struktur `Computer` die der Ausführung entsprechenden Werte enthalten. Als Rückgabe gibt `simcpu()` einen Vektor mit einer *aufsteigend nach Adresse geordneten* sortierten Liste von *geänderten* – d.h. vom Ausgangszustand abweichenden – Speicherstellen zurück. Das heißt, jedes Element des Vektors entspricht einer Speicherstelle, die durch ein Tupel dargestellt wird. Der erste Wert des Tupels ist die Adresse der Speicherstelle und der zweite Wert des Tupels ist der Wert der Speicherstelle. Es wird nur der Endzustand dokumentiert, so dass jede Adresse maximal einmal vorkommen kann.

[4] Dies entspricht etwa einem inversen Little-Endian-Format, vgl. `https://de.wikipedia.org/wiki/Byte-Reihenfolge\#Little-Endian-Format`

### Einschränkungen

Bei der Implementation unterliegen Sie einigen Einschränkungen, d. h. Sie dürfen bestimmte Programmierkonstrukte **nicht** nutzen:[5]

- Sie dürfen **keinen** unsicheren Code schreiben (Schlüsselwort `unsafe`)!
- Ihr Code darf **nichts** ausgeben (`println`, etc.).
- Schreiben Sie **keine** `main()`-Funktion![6]
- Neben selbst definierten Funktionen und den Standard-Operatoren dürfen Sie die folgenden Methoden/Operatoren aus der Standard-Bibliothek nutzen (und die entsprechenden Traits ableiten (`derive`)):
  - alle Methoden/Operatoren welche durch `Copy`, `Clone`, `Eq` und `PartialEq` bereitgestellt werden;
  - alle Methoden des Typs Vektor (`Vec`) sowie die Methoden der Traits welche für `Vec` implementiert sind;
  - alle Methoden des Traits `Iterator`.

  Sie dürfen **keine** anderen Funktionen/Methoden/Operatoren aus Bibliotheken nutzen.
- Sie dürfen **keine** Änderungen an den Definitionen der vorgegebenen Datenstrukturen/Traits (`Computer`, `Registers` und `Cpu`) vornehmen.

## 4. Hinweise zur Aufgabe

Die folgenden Schritte können Ihnen helfen, die Aufgabe zu bearbeiten. Falls Sie keine Hilfestellung benötigen, können Sie diesen Abschnitt überspringen.

- Schreiben Sie Hilfsfunktionen.
- Iteratoren können hilfreich sein.
- Testen Sie Ihren Code (siehe nächster Abschnitt).
- Spielen Sie ein paar Beispiele mit Stift und Papier durch.

### Testen

Beachten Sie, dass Ihre Abgabe kein vollständiges Programm ist, und dass sie keine `main()`-Funktion enthalten soll. Da Sie für den Test Ihrer Funktion jedoch ein ausführbares Programm benötigen, können Sie zum Testen z.B. den Code aus der beigefügten Datei `cpu.rs` nutzen (es wird nicht alles getestet). Beachten Sie aber, dass Ihr Abgabecode **nicht** den Testcode enthalten darf!

## 5. Einreichung

[5] Wenn Sie eines der folgenden Konstrukte nicht kennen, dann ignorieren Sie es.
[6] Sie dürfen diese natürlich schreiben, aber sie soll nicht in der Abgabe enthalten sein.

- Ihr Abgabe-Code muss auf den Rechnern des URZ-Pools übersetzbar sein mit dem Befehl `rustc --crate-type lib --emit obj cpu.rs`.
- Erstellen Sie ein Archiv in Form einer **.zip-Archivdatei**, welche **ausschließlich** Ihre Lösungsdatei `cpu.rs` enthält.
- Fügen Sie dem Archiv **keine** weiteren Dateien oder Ordner hinzu.
- Reichen Sie Ihre Lösung unter folgendem Link ein: `https://bildungsportal.sachsen.de/opal/auth/RepositoryEntry/41245343744/CourseNode/1757903993508949003`
- Sie erhalten eine Mail von OPAL als Bestätigung für die Einreichung Ihrer Abgabe. Bitte bewahren Sie die Mail auf.
- Falls Sie eine bestehende Lösung aktualisieren wollen, laden Sie bitte eine neue **.zip-Archivdatei** hoch. Wir bewerten Ihre letzte Abgabe (die Benennung spielt keine Rolle).

06.01.2026